TE4

Proposed Architecture:

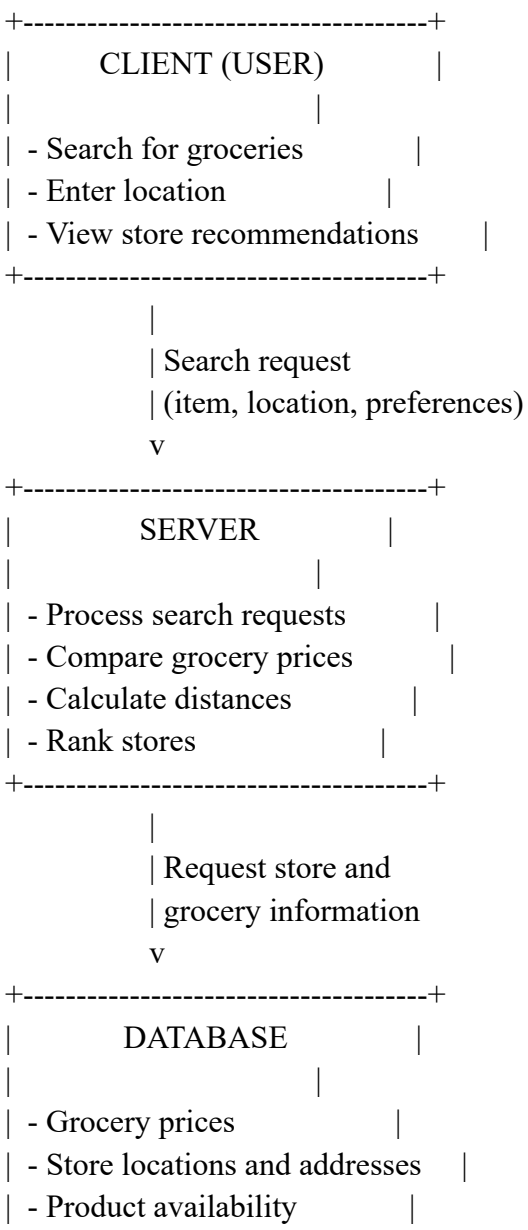
1. Name of the Architecture Pattern:

Client-Server Architecture:

2. Why It Fits the Application

A client-server architecture will allow users to access data from servers, such as grocery prices and store locations, to determine the best place to shop.

3. Diagram Showing the Major Components

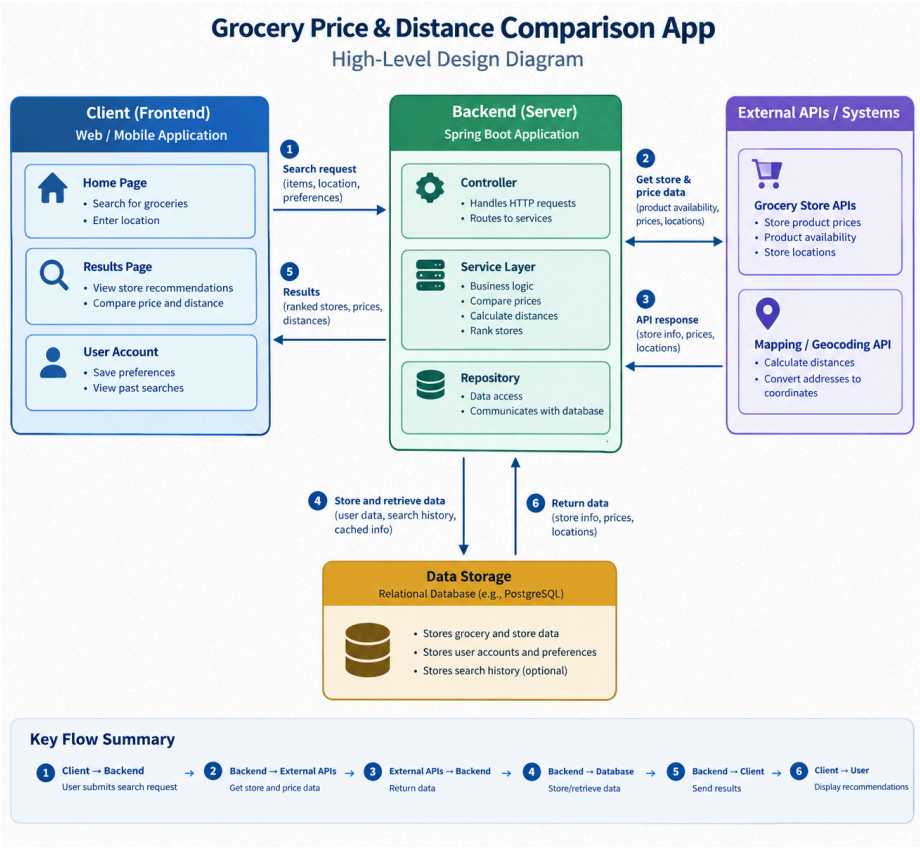


+-----------------------------------+

4. Direction of Communication Among Components

1. Client to Server: The user submits a grocery search, location, and preferences through the application.
2. Server to Database: The server requests grocery prices, store locations, and product availability information.
3. Database to Server: The database returns the requested grocery and store information.
4. Server Processing: The server calculates distances, compares prices, and ranks stores based on the user's preferences.
5. Server to Client: The server sends the ranked results, including store names, prices, and distances, back to the client.
6. Client to User: The client displays the results so the user can choose the store that best meets their needs.

High-Level Design Diagram:



Design Pattern: Adapter

Status: Planned

1. Problem the Pattern Solves

The Adapter design pattern will allow our grocery comparison application to retrieve grocery prices and product availability from different grocery store APIs, even if those APIs use different data formats or interfaces.

For example, one grocery store API might return prices in a JSON field called price, while another might use unitPrice. Without an adapter, the backend would need separate logic for handling each store's API throughout the application.

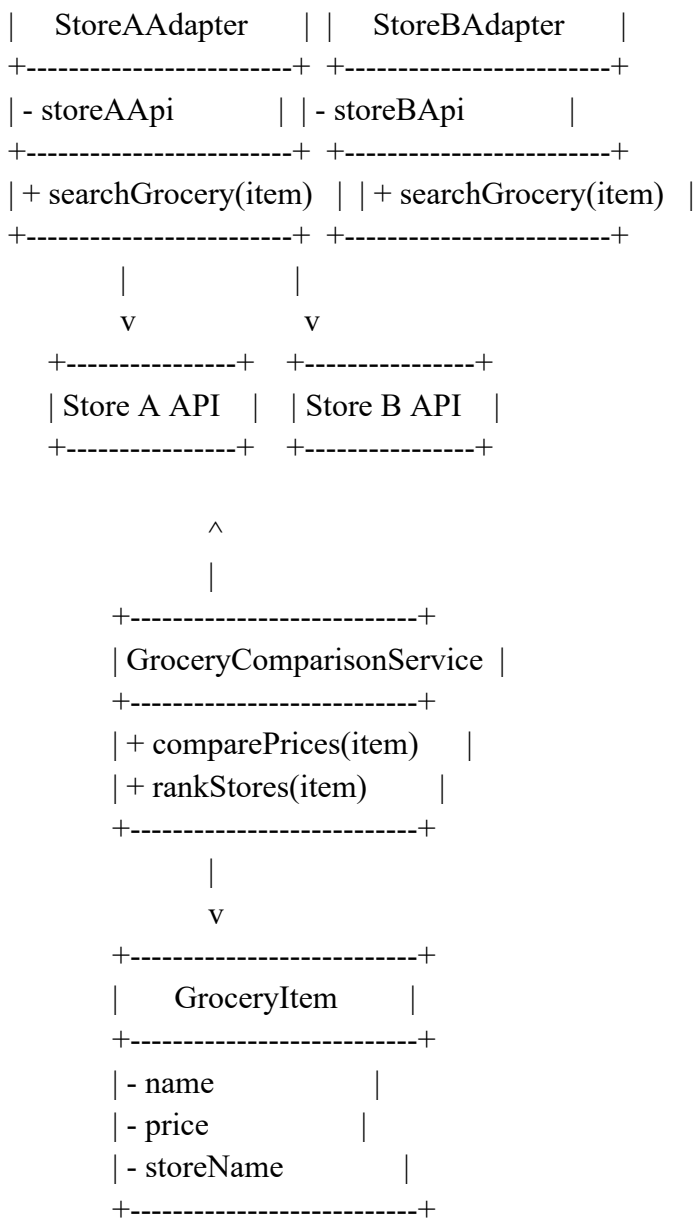
The Adapter pattern solves this problem by converting each store's data into a common format that the application can understand.

2. Classes and Components Involved

- GroceryStoreAdapter: A common interface that defines how the application retrieves grocery information.
- StoreAAdapter: Connects to Store A's API and converts its response into the application's standard format.
- StoreBAdapter: Connects to Store B's API and converts its response into the same standard format.
- GroceryComparisonService: Uses the adapters to retrieve grocery data, compare prices, and rank stores.
- GroceryItem: A standardized representation of a grocery item, including its name, price, and store information.

3. UML Class Diagram





4. Why This Pattern Is Better Than One Large Component

The Adapter pattern separates the code for communicating with individual grocery store APIs from the application's main comparison logic. Instead of placing every store's API requests, response formats, and price comparisons in one large component, each adapter handles the differences specific to its store.

This makes the application easier to maintain, test, and expand. If the team adds another grocery store, it can create a new adapter without rewriting the entire comparison service. It also makes it easier to handle changes to an individual store's API.

5. Planned Implementation

Our team intends to define a common GroceryStoreAdapter interface and implement a separate adapter for each supported grocery data source. Each adapter will retrieve the relevant grocery information and convert it into a standardized GroceryItem representation. The GroceryComparisonService will then use these standardized results to compare prices, calculate distances using location data, and rank stores according to the user's preferences.

4. Design Decisions

Decision 1: Using Client-Server Architecture

Options Considered: Our team considered building the application as a single standalone application or using a client-server architecture.

Selected Option: Client-server architecture.

Reasoning: We selected client-server architecture because it separates the user interface from the backend processing. The client allows users to search for groceries and view results, while the server handles price comparisons, distance calculations, and store rankings. This separation makes the application easier to maintain and allows multiple users to access the service.

Decision 2: Using the Adapter Design Pattern

Options Considered: Our team considered writing separate API-handling logic directly inside the grocery comparison service or using the Adapter design pattern to standardize information from different grocery stores.

Selected Option: The Adapter design pattern.

Reasoning: We selected the Adapter pattern because different grocery stores may provide prices and product information in different formats. Adapters will convert these responses into a common format that the application can process. This will make it easier to add new grocery stores and maintain existing integrations without rewriting the main comparison logic.